

Structured Programming – Control Structures

Computer Engineering 1

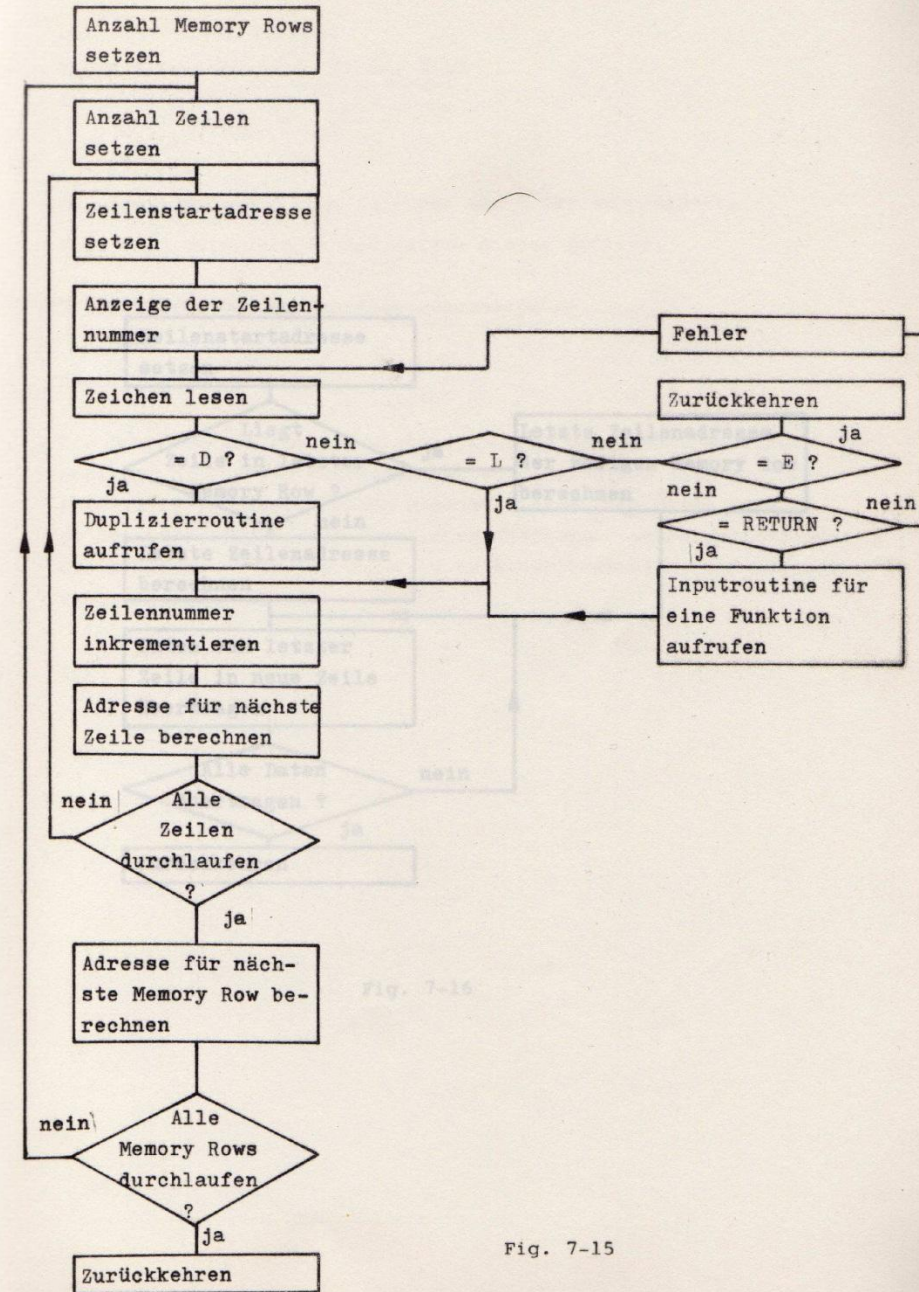
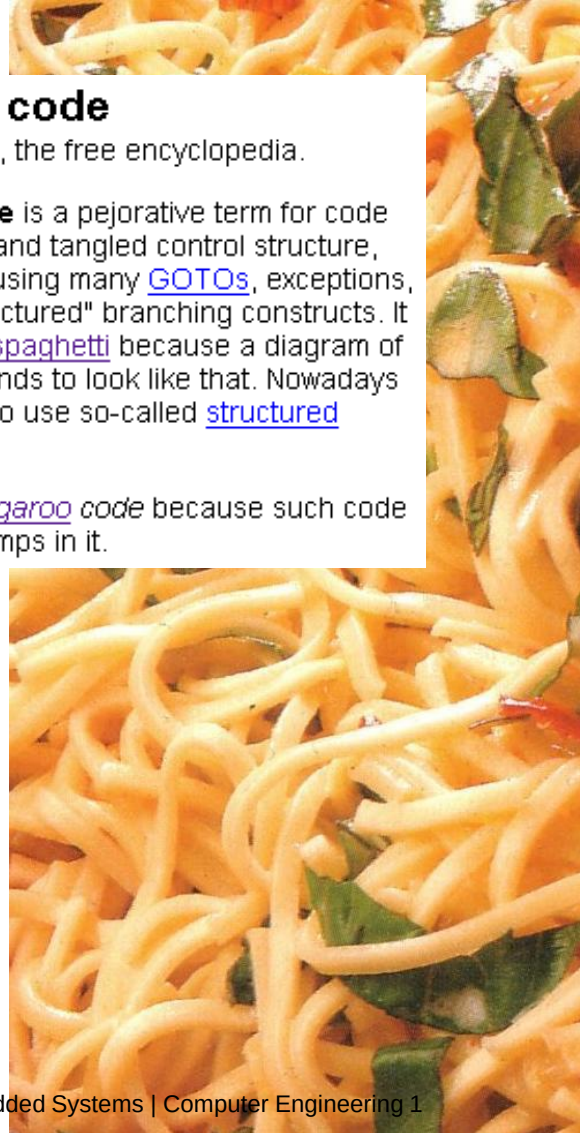
Motivation

Spaghetti code

From Wikipedia, the free encyclopedia.

Spaghetti code is a pejorative term for code with a complex and tangled control structure, especially one using many [GOTOs](#), exceptions, or other "unstructured" branching constructs. It is named after [spaghetti](#) because a diagram of program flow tends to look like that. Nowadays it is preferable to use so-called [structured programming](#).

Also called [kangaroo code](#) because such code has so many jumps in it.



Agenda

- **Structured Programming**
- **Selection**
 - if – then - else
- **Loops**
 - Do – While
 - While
 - For
- **Switch Statements**

Learning Objectives

At the end of this lesson you will be able

- to explain the basic concepts of structured programming
- to enumerate and explain the basic elements of a structogram
- to comprehend how a C-compiler implements control structures in assembly language
 - if-then-else
 - do-while loops
 - while loops
 - for loops
 - switch statements
- to program basic structograms in assembly language

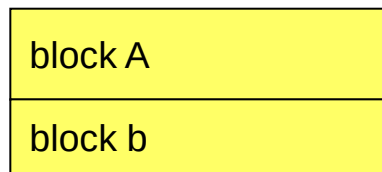
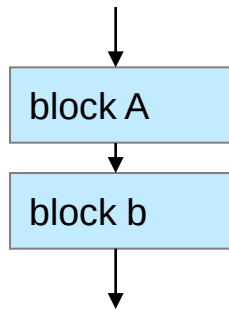
Why Structured Programming ?

- **Rules for the structure of a program**
 - Patterns for control structures
 - Sequence
 - Selection if - then - else
 - Iteration / Loop for, while, do - while
 - Compilers generate code-blocks based on these patterns
- **Supports program development**
 - Clarity
 - Documentation
 - Maintenance
 - Allows to program on a higher level of abstraction

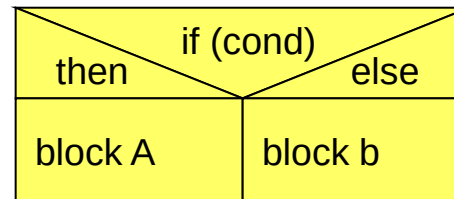
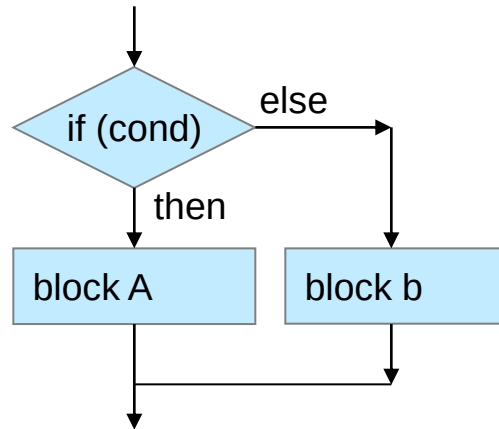
Structured Programming

Program flow can be represented with three elements

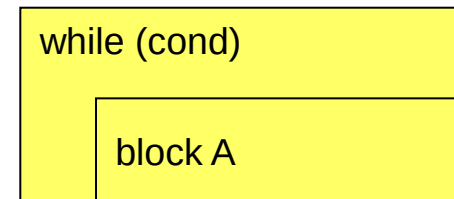
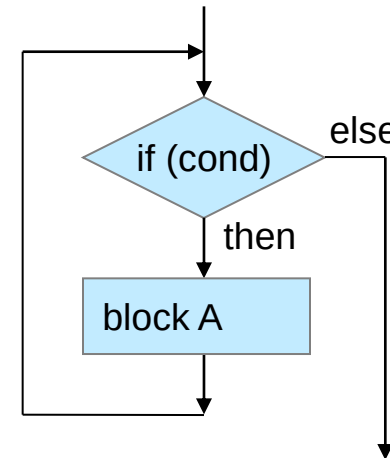
Sequence



Selection



Iteration / loop

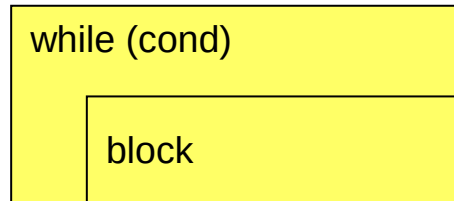


Structured Programming

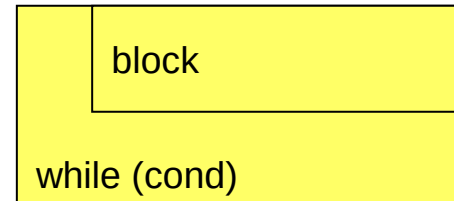
Further Structograms

– Iteration

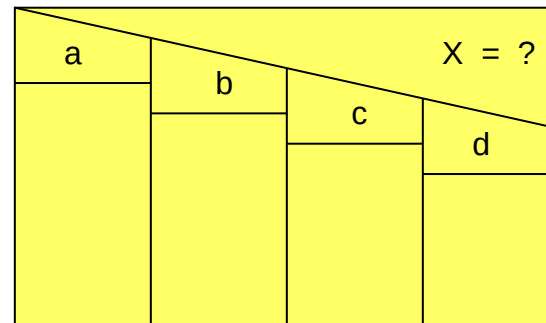
pre-test loop



post-test loop



– Switch statement (case)

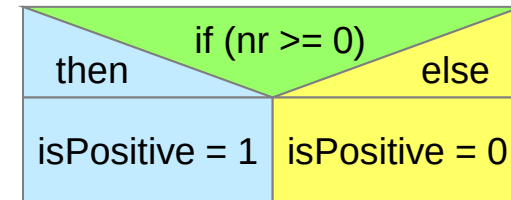
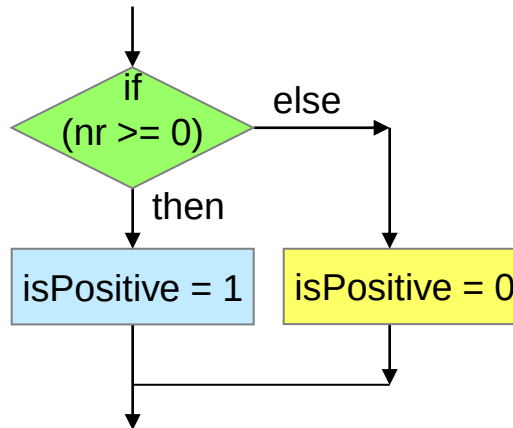


Selection

if – then – else

```
int32_t nr, isPositive;  
...
```

```
if (nr >= 0) {  
    isPositive = 1;  
}  
else {  
    isPositive = 0;  
}
```



Selection

...if – then – else

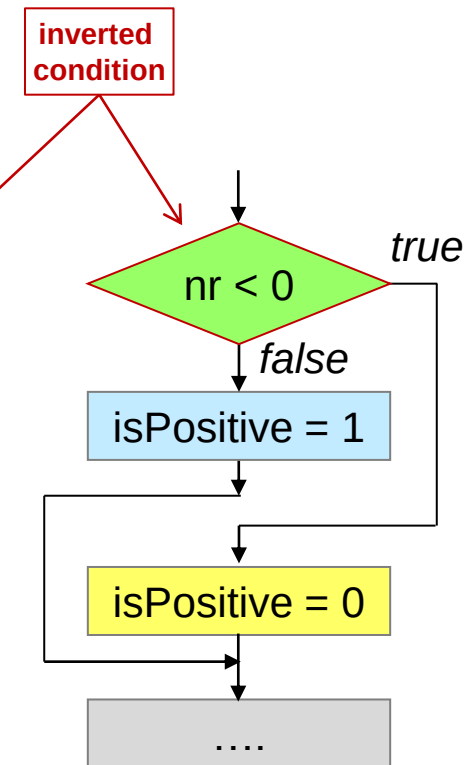
- Compiler translates selection into assembly code
 - uses conditional and unconditional jumps

```
int32_t nr;  
int32_t isPositive;  
...  
if (nr >= 0) {  
    isPositive = 1;  
}  
else {  
    isPositive = 0;  
}
```



Assume: nr in r1
isPositive in r2

	cmp	r1, #0x00
	b	else
	movs	r2, #1
	b	end_if
else		
	movs	r2, #0
end_if		
		

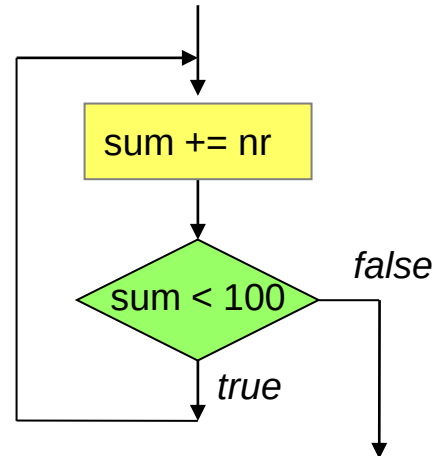


Loops

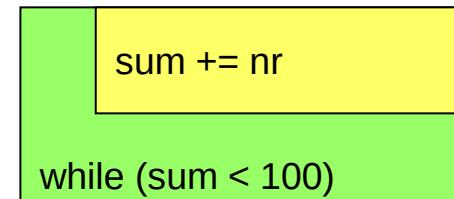
Do-While Loops

```
int32_t nr;  
int32_t sum;  
...  
sum = 0;
```

```
do {  
    sum += nr;  
} while (sum < 100);
```



post-test loop



Loops

...Do-While Loops

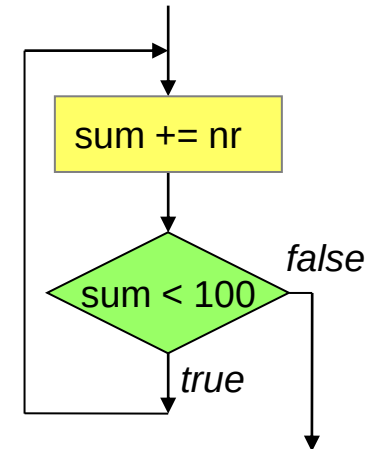
- Compiler translates post-test loop to assembly code

```
int32_t nr;  
int32_t sum;  
...  
sum = 0;
```

```
do {  
    sum += nr;  
} while (sum < 100);
```

Assume: nr in r1
sum in r2

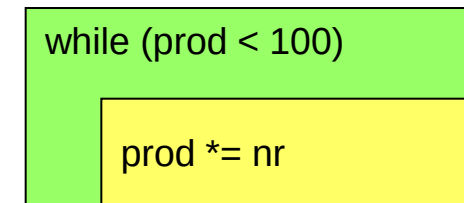
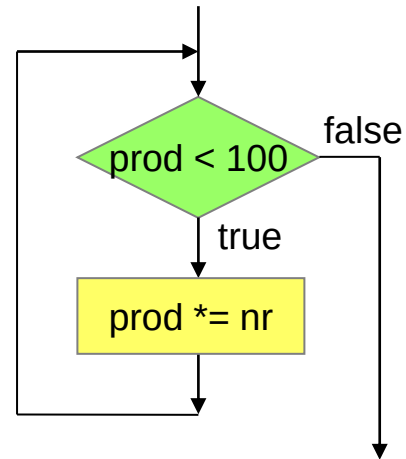
```
    movs    r2, #0  
loop  adds    r2, r2, r1  
      cmp    r2, #100  
      blt    loop  
      ....
```



Loops

While Loops

```
int32_t nr;  
int32_t prod;  
...  
prod = 1;  
while (prod < 100) {  
    prod *= nr;  
}
```



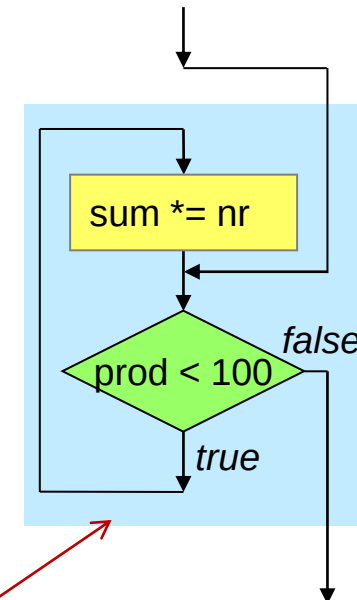
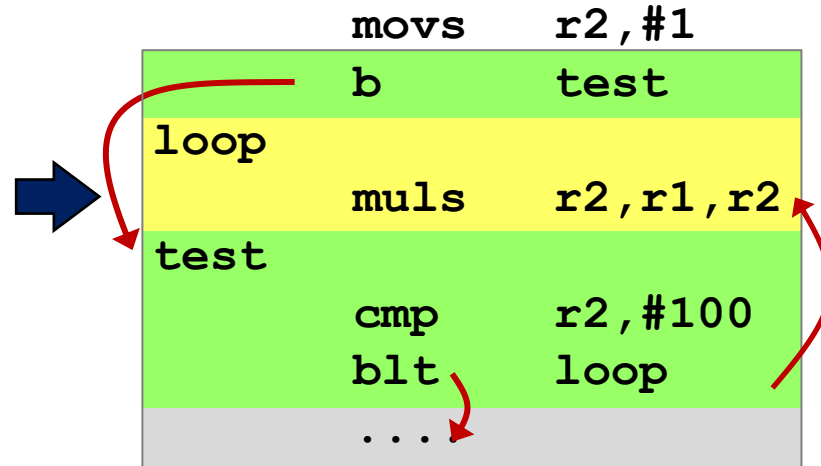
Loops

...While Loops

- Compiler translates pre-test loop to assembly code
 - Re-using structure of do-while (pre-test loop)

```
int32_t nr;  
int32_t prod;  
...  
prod = 1;  
while (prod < 100) {  
    prod *= nr;  
}
```

Assume: nr in r1
prod in r2



re-use do-while

Loops

For Loops

- For Loops are converted into While Loops
 - break/continue statements require special treatment

```
for (init-expr; test-expr; update-expr)  
    body-block
```

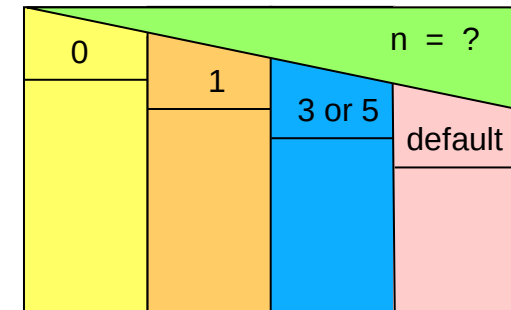
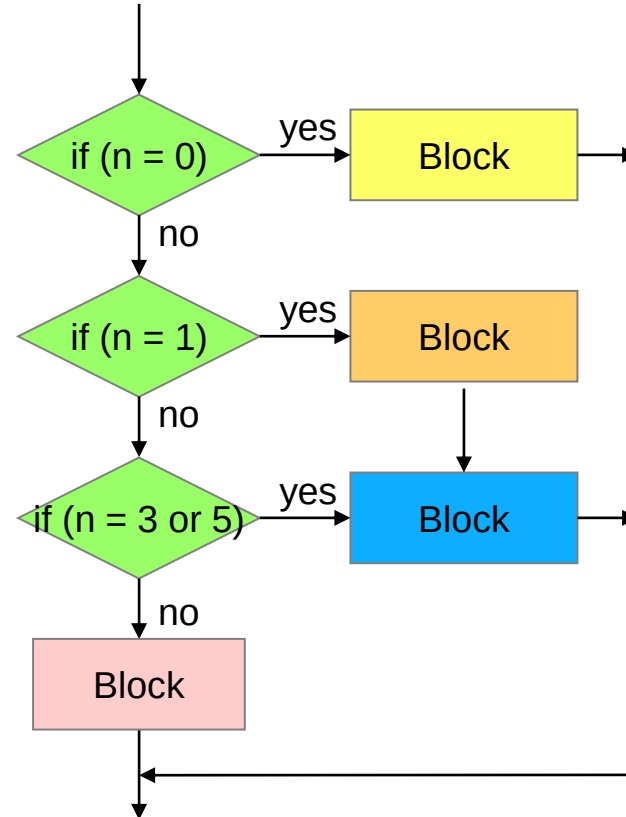


```
init-expr;  
while (test-expr) {  
    body-block  
    update-expr;  
}
```

Switch Statements

```
uint32_t result, n;
```

```
switch (n) {  
  case 0:  
    result += 17;  
    break;  
  case 1:  
    result += 13;  
    //fall through  
  case 3: case 5:  
    result += 37;  
    break;  
  default:  
    result = 0;  
}
```



Structogram without fall-through

Switch Statements

Jump Table

```
uint32_t result, n;  
switch (n) {  
case 0:  
    result += 17;  
    break;  
case 1:  
    result += 13;  
    //fall through  
case 3: case 5:  
    result += 37;  
    break;  
default:  
    result = 0;  
}
```

Assume: n in r1
result in r2

```
jump_table:  
    .word    case_0  
    .word    case_1  
    .word    case_default  
    .word    case_3_5  
    .word    case_default  
    .word    case_3_5
```

```
.equ NR_CASES, 6  
case_switch:  
    cmp     r1, #NR_CASES  
    bhs     case_default  
    lsls    r1, #2    ; * 4  
    ldr     r7, =jump_table  
    ldr     r7, [r7, r1]  
    bx      r7
```

```
.thumb_func  
case_0:  
    adds    r2, r2, #17  
    b       end_sw_case
```

```
.thumb_func  
case_1:  
    adds    r2, r2, #13
```

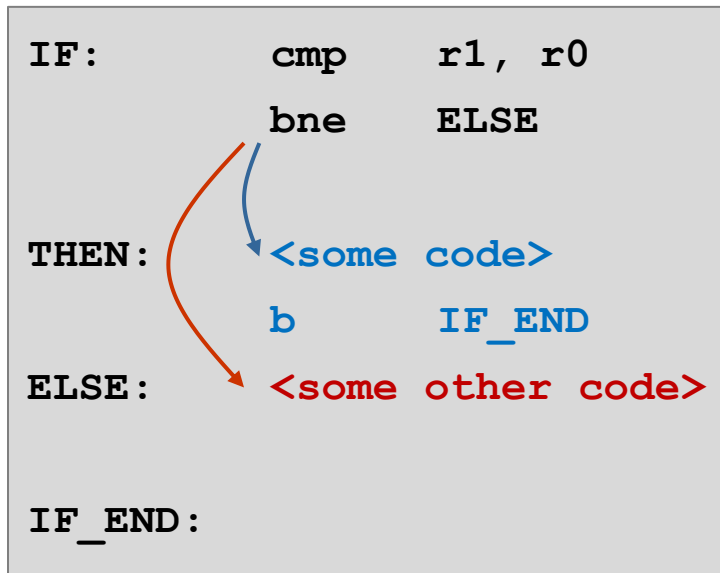
```
.thumb_func  
case_3_5:  
    adds    r2, r2, #37  
    b       end_sw_case
```

```
.thumb_func  
case_default:  
    movs    r2, #0  
end_sw_case:    ...
```

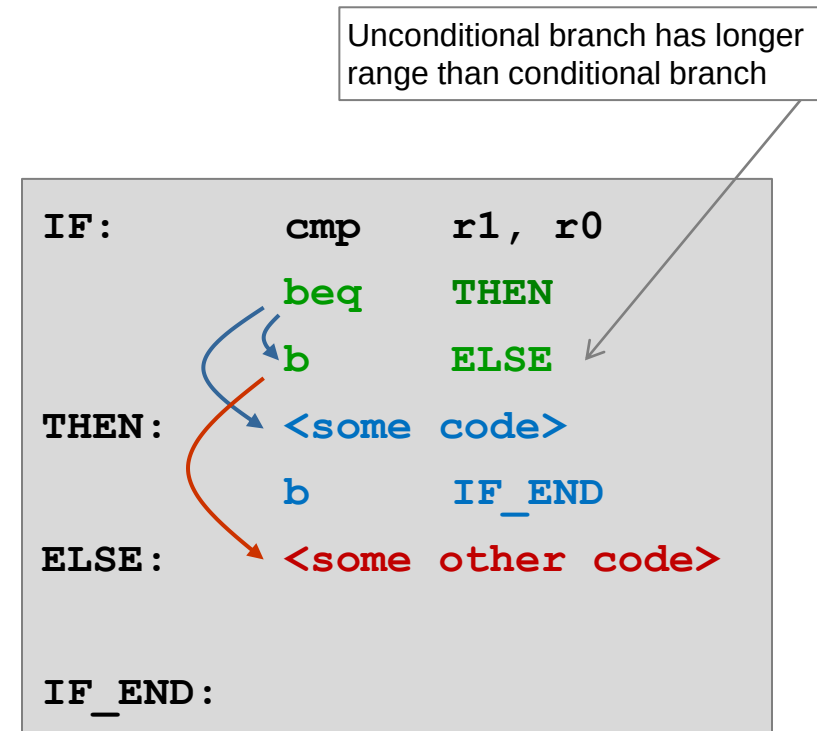
Limitations of Conditional Branches

- **Limited range of -256..254 Bytes**

- Example



Simple code for the case when
<some code> is short

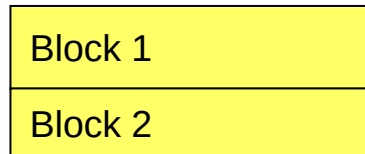
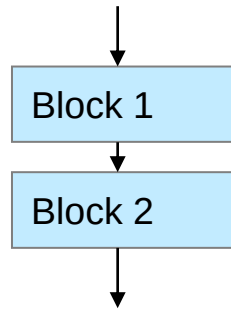


Code requires additional branch in
case when <some code> is too long

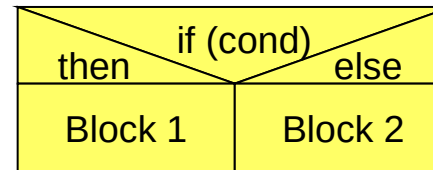
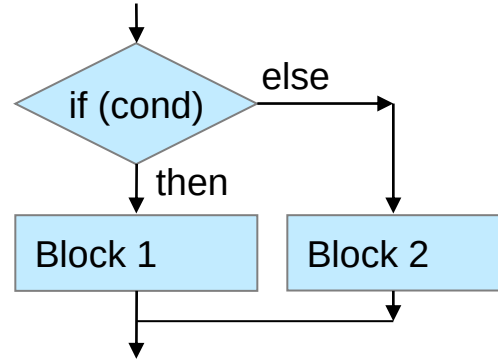
Conclusion

- Program flow can be represented with three elements

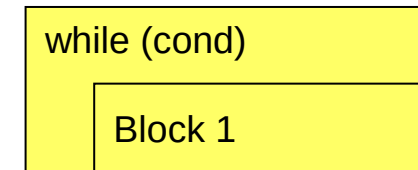
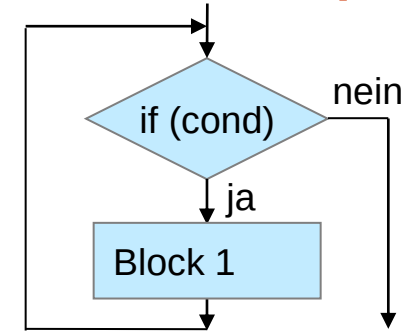
Sequence



Selection



Iteration/loop



- High level programming language provides these control structures
- Compiler translates control structures to assembly using conditional and unconditional jumps